Lab 1: Back-propagation

廖偉菖 314554029

March 14, 2026

1 Introduction

This lab focuses on the fundamental concepts of deep learning by implementing a multi-layer perceptron (MLP) entirely from scratch using only Python and NumPy. We are tasked with building a neural network featuring two hidden layers, implementing both forward propagation and backpropagation for calculating gradients. The primary goal is to solve two classification problems—a linearly separable dataset (Linear) and a non-linear dataset (XOR)—demonstrating how backpropagation updates weights to achieve an accuracy exceeding 90%.

2 Implementation Details

2.1 Sigmoid Function

For the baseline model, the activation function chosen for all layers is the Sigmoid function, which maps any real value into the range $(0, 1)$. Its mathematical definition is:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

The derivative of the sigmoid function, which is crucial for the backpropagation step, is elegantly expressed in terms of the sigmoid output itself:

$$\sigma'(x) = \sigma(x) \times (1 - \sigma(x)) \tag{2}$$

Implementation: To ensure numerical stability and prevent overflow errors, the input array is safely bounded using `np.clip` before applying the mathematical formula.

```
def forward(self, inputs):
    inputs = np.clip(inputs, -500, 500)
    self.outputs = 1.0 / (1.0 + np.exp(-inputs))
    return self.outputs

def backward(self, grad_output, lr):
    grad_activation = self.outputs * (1.0 - self.outputs)
    return grad_output * grad_activation
```

2.2 Neural Network Architecture

The baseline architecture is constructed as a fully connected feedforward neural network comprising:

- **Input Layer:** 2 neurons (representing x_1 and x_2).
- **Hidden Layer 1:** 8 neurons with Sigmoid activation.
- **Hidden Layer 2:** 8 neurons with Sigmoid activation.
- **Output Layer:** 1 neuron with Sigmoid activation for binary classification.

The forward pass is defined mathematically by the following sequence:

$$\begin{aligned} Z_1 &= \sigma(XW_1 + b_1) \\ Z_2 &= \sigma(Z_1W_2 + b_2) \\ \hat{y} &= \sigma(Z_2W_3 + b_3) \end{aligned}$$

Implementation: The structure is realized via a `Sequential` class, which iteratively passes the output of the current layer as the input to the subsequent layer.

```
def forward(self, inputs):
    x = inputs
    for layer in self.layers:
        x = layer.forward(x)
    return x
```

2.3 Backpropagation

Backpropagation uses the chain rule to recursively calculate the gradient of the Mean Squared Error (MSE) loss function with respect to the network's weights and biases. The loss is defined as:

$$L(\theta) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 \tag{3}$$

For hidden layers, the error propagates backward via the chain rule. Weights and biases are then updated via Gradient Descent:

$$W_i \leftarrow W_i - \eta\Delta W_i \quad \text{where} \quad \Delta W_i = Z_{i-1}^T\delta_i \tag{4}$$

Implementation: These equations are efficiently vectorized using `np.dot` to compute the gradients and update the parameters in-place.

```
def backward(self, grad_output, lr):
    grad_inputs = np.dot(grad_output, self.weights.T)
    grad_weights = np.dot(self.inputs.T, grad_output)
    grad_bias = np.sum(grad_output, axis=0, keepdims=True)

    self.weights -= lr * grad_weights
    self.bias -= lr * grad_bias
    return grad_inputs
```

3 Experimental Results

3.1 Linear Dataset

The network was trained on the Linear dataset using a learning rate ($\eta = 0.1$).

- **Final Accuracy achieved:** 100.00%

```
Training on Linear Dataset
epoch     0 loss : 0.2487628793
epoch  5000 loss : 0.0028143428
epoch 10000 loss : 0.0009416404
epoch 15000 loss : 0.0004749174
epoch 19999 loss : 0.0002938041
```

Figure 1: Terminal Screenshot: Training Loss (Linear)

```
Iter93 | Ground truth: 1.0 | prediction: 1.00000 |
Iter94 | Ground truth: 0.0 | prediction: 0.00465 |
Iter95 | Ground truth: 1.0 | prediction: 0.99937 |
Iter96 | Ground truth: 0.0 | prediction: 0.00017 |
Iter97 | Ground truth: 1.0 | prediction: 1.00000 |
Iter98 | Ground truth: 0.0 | prediction: 0.00027 |
Iter99 | Ground truth: 1.0 | prediction: 1.00000 |
Iter100 | Ground truth: 1.0 | prediction: 0.99999 |
loss=0.00039 accuracy=100.00%
```

Figure 2: Terminal Screenshot: Testing Results and Accuracy (Linear)

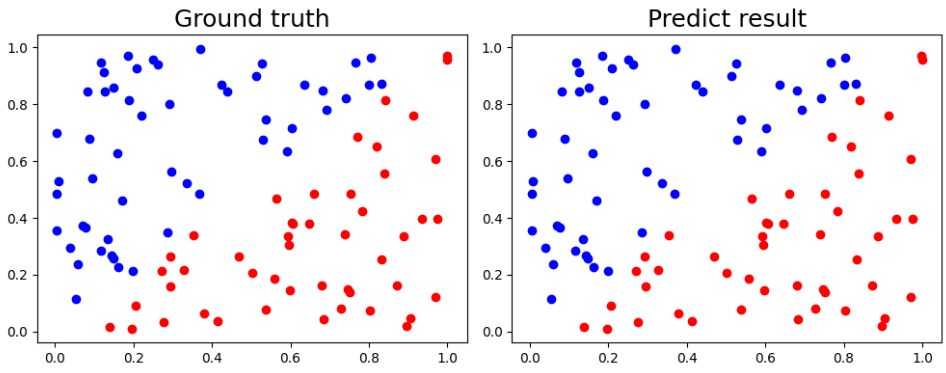

Figure 3: Ground Truth vs Prediction for the Linear Dataset

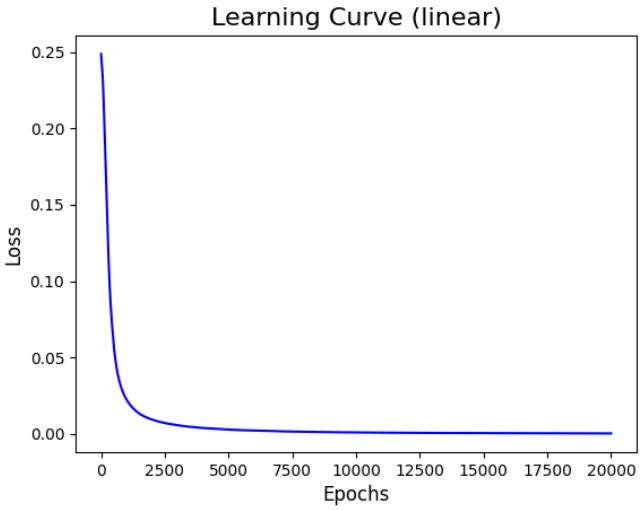

Figure 4: Learning Curve for the Linear Dataset

3.2 XOR Dataset

The XOR dataset poses a more challenging non-linear problem. The model successfully converged using the architecture ($2 \to 8 \to 8 \to 1$) with a higher learning rate ($\eta = 0.5$).

- **Final Accuracy achieved:** 100.00%

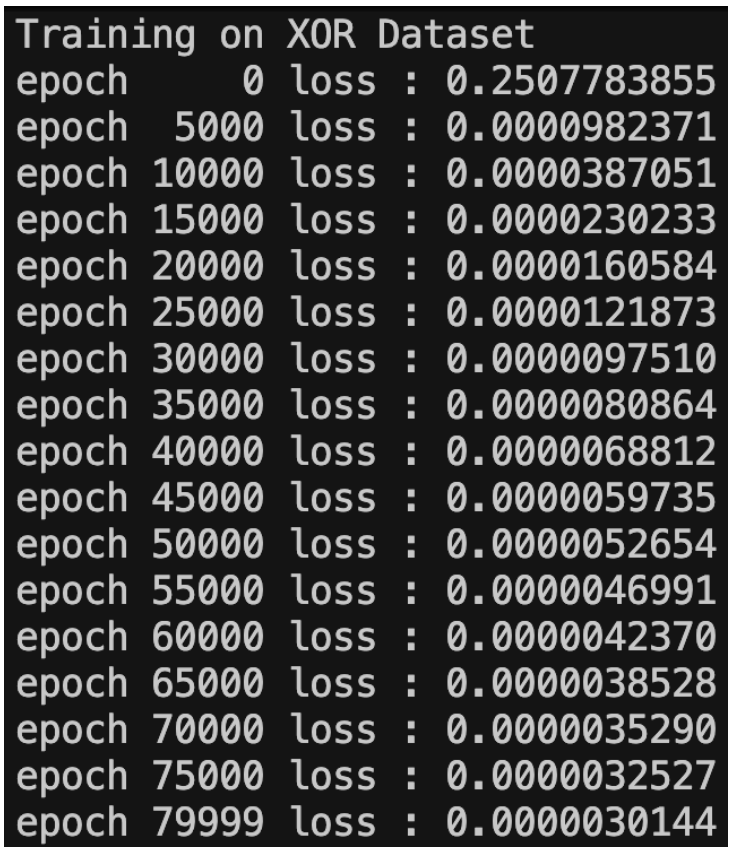

Figure 5: Terminal Screenshot: Training Loss (XOR)

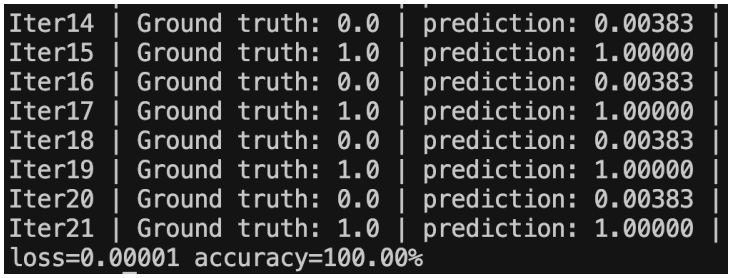

Figure 6: Terminal Screenshot: Testing Results and Accuracy (XOR)

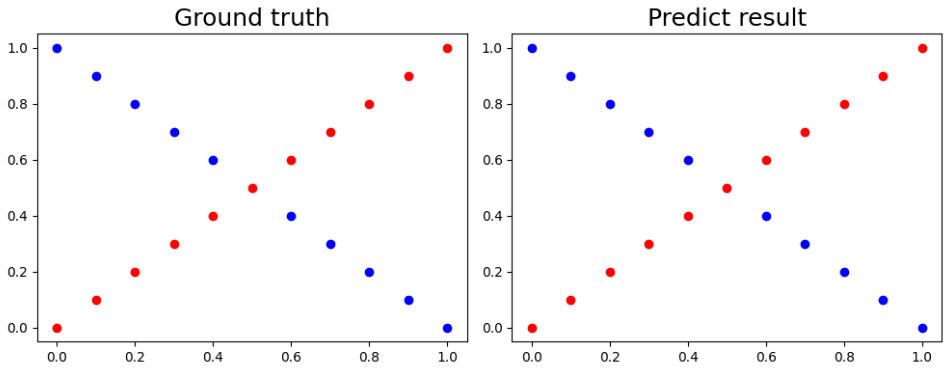

Figure 7: Ground Truth vs Prediction for the XOR Dataset

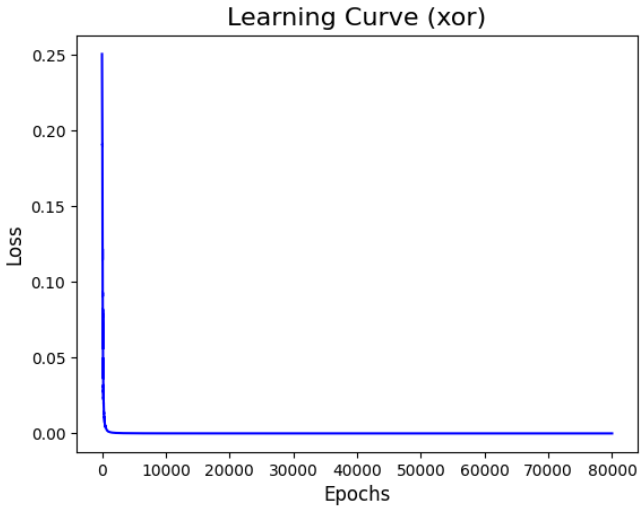

Figure 8: Learning Curve for the XOR Dataset

4 Discussion

4.1 Learning Rate Experiments

Varying the learning rate significantly impacted convergence on both datasets:

- **Linear Dataset:** Because the problem is linearly separable, it is quite robust. It converges stably across a wide range of learning rates.
- **XOR Dataset:** When the learning rate was too high (e.g., > 5.0), the gradients became unstable, causing the loss curve to oscillate heavily and fail to converge. Conversely, a very small learning rate (e.g., 0.001) caused the network to learn extremely slowly, requiring significantly more epochs to cross the 90% accuracy threshold and sometimes getting stuck in local minima.

4.2 Hidden Units Experiments

- **Linear Dataset:** A model with 0 hidden layers (a simple logistic regression) can easily achieve 100% accuracy, proving that extra hidden units are not strictly necessary for simple linear boundaries.
- **XOR Dataset:** Reducing the number of hidden units to $(2 \to 2 \to 2 \to 1)$ resulted in the model frequently getting stuck in local minima. By increasing the architecture to 8 units per hidden layer, the representational capacity of the network improved vastly, practically ensuring convergence and 100% accuracy.

4.3 Without Activation Function

When the sigmoid activation functions were removed from the hidden layers (rendering them simple linear transformations):

- **Linear Dataset:** The network could still easily solve the dataset, as linear boundaries remain linear.
- **XOR Dataset:** The model completely failed (achieving around 50% accuracy). This demonstrates that without non-linearities, multiple layers collapse mathematically into a single linear transformation ($X \cdot W_1 \cdot W_2 \cdot W_3$), rendering the network incapable of learning complex spatial boundaries.

5 Questions

A. What is the purpose of activation functions?

Activation functions introduce non-linear properties to the network. Without non-linear activation functions, a deep neural network would behave identically to a standard single-layer linear regression model, regardless of how many layers it possesses. Activation functions allow networks to learn and approximate complex, high-dimensional, non-linear mappings from inputs to outputs.

B. What might happen if the learning rate is too large or too small?

Too large: The step size taken during gradient descent will overshoot the optimal local minimum of the loss function. This leads to mathematical instability, causing the loss to oscillate wildly or even diverge entirely.
Too small: The model makes incredibly tiny updates to its weights. While mathematically stable, the network will take an unreasonably long time to train and is highly prone to getting stuck in suboptimal local minima or saddle points before reaching convergence.

C. What is the purpose of weights and biases in a neural network?

Weights: Weights represent the strength and direction of the connection between two neurons. They dictate how much influence an input feature has on the output, scaling the data as it propagates through the network layers.
Biases: Biases act similarly to the intercept in a linear equation ($y = mx + b$). They allow the activation function curve to shift horizontally, which ensures the network can fit data distributions that do not pass through the origin $(0, 0)$ and prevents the neurons from always outputting 0 when inputs are 0.

6 Extra

To further explore the capabilities of neural networks, I implemented several advanced components beyond the baseline requirements, which are included in the source code:

A. Implement different optimizers

In addition to standard Stochastic Gradient Descent (SGD), I implemented the **Momentum** optimizer. By accumulating a velocity vector of past gradients, the momentum optimizer significantly accelerates convergence and helps the model push through shallow local minima.

```
if self.optimizer == 'momentum':
    self.v_w = self.momentum_factor * self.v_w - lr * grad_weights
    self.v_b = self.momentum_factor * self.v_b - lr * grad_bias
    self.weights += self.v_w
    self.bias += self.v_b
```

B. Implement different activation functions

To prevent the vanishing gradient problem, I implemented additional activation functions such as **ReLU**, **LeakyReLU**, and **Tanh**. For instance, the ReLU implementation efficiently computes gradients using boolean masking:

```
class ReLU(Module):
    def forward(self, inputs):
        self.inputs = inputs
        return np.maximum(0, inputs)

    def backward(self, grad_output, lr):
        grad_activation = (self.inputs > 0).astype(float)
        return grad_output * grad_activation
```

C. Implement convolutional layers

I implemented a **Conv2D** (2-Dimensional Convolutional) layer module from scratch. The module supports forward propagation by sliding a parameter matrix (kernel) across 2D spatial inputs to extract local features.

```
def forward(self, inputs):
    # Sliding window for 2D convolution
    for b in range(batch_size):
        for oc in range(self.weights.shape[0]):
            for i in range(out_h):
                for j in range(out_w):
                    region = inputs[b, :, i:i+self.kernel_size, j:j+
                        self.kernel_size]
                    outputs[b, oc, i, j] = np.sum(region * self.
                        weights[oc]) + self.bias[oc]
    return outputs
```